

Respuestas Laboratorio 1: Git

Nombre y apellidos: Alejandro Marti Cadavid

Nombre del repositorio: oracle-database-lab

URL del repositorio (GitHub): <https://github.com/amarti48/oracle-database-lab.git>

1. Lecciones aprendidas

- He entendido que un repositorio de Git no rastrea carpetas vacías por lo que tienes que usar archivos como .gitkeep para estructurar bien el proyecto desde el principio.

-He aprendido a identificar y diferenciar con "git status" cuando un archivo está en el Working Directory, en la Staging Area o cuando está guardado definitivamente en el Local Repository tras un commit.

-He comprendido que crear una rama no duplica los archivos de las carpetas físicas en el disco, sino que mueve el puntero en el historial interno de la carpeta oculta.

-Los mensajes de conflicto, son puramente marcas textuales que manda Git para que yo decida que hacer.

2. Respuestas a las preguntas de comprobación

1. ¿Cuál es la diferencia entre Working Directory, Staging Area y Local Repository? Da un ejemplo de un archivo pasando por las tres.

-Working Directory: Es la carpeta del proyecto tal como la ves y editas en tu disco.

-Staging Area: Es la zona intermedia donde preparas y confirmas exactamente qué archivos irán en el próximo commit.

-Local Repository: Es el historial histórico de commits guardado internamente en la carpeta .git.

Ejemplo: Creo un archivo README.md (está en el Working Directory). Ejecuto git add README.md (pasa a la Staging Area). Ejecuto git commit -m "docs: inicial" (se guarda permanentemente en el Local Repository).

2. Si modificas un archivo pero no haces git add, ¿aparece ese cambio en tu próximo commit? Explica por qué.

No, el cambio no aparecerá. El comando git commit únicamente empaqueta y confirma los archivos que previamente han sido movidos a la Staging Area mediante git add

3. ¿Por qué git status no mostraba las carpetas vacías que creaste en la Parte C? ¿Qué truco usamos para solucionarlo?

Porque Git no versiona carpetas sino que versiona archivos. El truco profesional utilizado es crear un archivo vacío oculto dentro de cada carpeta, para obligar a Git a rastrear el directorio

4. Explica con tus palabras qué es HEAD.

Head es un puntero interno de Git que te indica en que branch o rama y commit te encuentras.

5. ¿Qué diferencia hay entre crear una branch con git switch -c y crear una carpeta nueva con mkdir? ¿Cómo lo comprobamos en la Parte G?

El comando mkdir crea una carpeta física real en el sistema de archivos, por el contrario, git switch -c crea un puntero sin duplicar ninguna carpeta física.

6. Durante el conflicto de la Parte H, ¿qué representaba el contenido entre <<<<<< HEAD y =====? ¿Y entre ===== y >>>>>>?

El contenido entre <<<<<< HEAD y ===== representa la versión del código que ya tenías en tu rama actual (en este caso, main). El contenido situado entre ===== y >>>>>> fix/readme-subtitle representa la versión entrante que venía de la rama que intentabas fusionar.

7. ¿Por qué NO se debe hacer git commit --amend sobre un commit que ya se subió con git push?

Porque el comando --amend reescribe la historia de Git creando un commit completamente nuevo con un código *hash* distinto, haciendo desaparecer el anterior. Si el commit original ya estaba en GitHub y alguien más lo ha descargado, esto provocará historiales divergentes muy dolorosos de resolver para el equipo.

8. Si borras por accidente la carpeta .git de tu proyecto, ¿qué se pierde exactamente? ¿Se pierde también el código fuente que está en el disco?

Se pierde toda la base de datos interna de Git, es decir, todo el historial del repositorio, los commits y las ramas. Sin embargo, **no** se pierde el código fuente físico; los archivos de tu Working Directory (el código en el que estabas trabajando) se mantienen intactos en el disco.

9. Explica con tus propias palabras la diferencia entre Git y GitHub, sin usar la palabra "nube".

Git es el programa instalado en mi ordenador que realiza el control de versiones y gestiona el historial de mis archivos de forma autónoma. GitHub es una plataforma web externa que sirve para alojar remotamente los repositorios creados con Git, permitiendo centralizarlos, visualizar el código en el navegador y colaborar en equipo.

10. ¿Por qué no se debe subir un archivo .env con contraseñas reales a un repositorio, aunque el repositorio sea privado?

Porque las contraseñas quedarían registradas en el historial de forma permanente. Cualquier usuario con acceso de lectura al repositorio, o herramientas automatizadas conectadas al mismo, podrían ver y extraer esas credenciales reales, suponiendo un grave riesgo de seguridad.

3. Capturas de pantalla

```
aleja@DESKTOP-QI5HJ62 MINGW64 ~/oracle-database-lab (main)
$ git log --oneline --graph --all
* 3f90ea1 (HEAD -> main, origin/main) merge: resolve README title conflict
|
| * ff48f2d (origin/fix/readme-subtitle, fix/readme-subtitle) docs: rename project title (academic version)
| * ba3f189 (origin/fix/readme-title, fix/readme-title) docs: rename project title (training edition)
|/
| * 0665668 Merge branch 'feature/customer-search'
|/
| * f743423 (feature/customer-search) docs: draft customer search notes
| * 9200a61 chore: add directory structure with .gitkeep
|/
* a8326cd docs: fix typo in schema notes
* 5df9d9b docs: add customer schema notes
* 0a4a503 docs: add initial project documentation

aleja@DESKTOP-QI5HJ62 MINGW64 ~/oracle-database-lab (main)
$ |
```

```
PS C:\Users\aleja> cmd //c tree
Microsoft Windows [Versión 10.0.26200.9457]
(c) Microsoft Corporation. Todos los derechos reservados.
```

```
aleja@DESKTOP-QI5HJ62 MINGW64 ~/oracle-database-lab (main)
$ git remote -v
origin https://github.com/amarti48/oracle-database-lab.git (fetch)
origin https://github.com/amarti48/oracle-database-lab.git (push)

aleja@DESKTOP-QI5HJ62 MINGW64 ~/oracle-database-lab (main)
$ |
```

oracle-database-lab Public

Pin Watch 0 Fork 0 Star 0

main 3 Branches 0 Tags

Go to file

Add file

Code

amarti48 merge: resolve README title conflict 3f90ea1 · 34 minutes ago 9 Commits

database	chore: add directory structure with .gitkeep	39 minutes ago
docs	docs: draft customer search notes	45 minutes ago
playbooks	chore: add directory structure with .gitkeep	39 minutes ago
runbooks	chore: add directory structure with .gitkeep	39 minutes ago
scripts	chore: add directory structure with .gitkeep	39 minutes ago
README.md	merge: resolve README title conflict	34 minutes ago

README

Oracle Database Lab (Training Edition — Academic Version)

Training repository for Oracle Database administration, testing, change management and Git workflows. Name: Alejandro Marti Professor: Richard Aviles Lopez

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1

amarti48